Universidad de San Carlos de Guatemala
Centro Universitario de Occidente
División de Ciencias de la Ingeniería
Lenguajes Formales y de Programación



Manual Técnico Proyecto 2 Lenguajes

202330209 Milton Efrén Ajxup Ramos

Quetzaltenango, octubre 2026

Índice

Índice —---------- 2

Introducción —---------- 3

Manual Técnico —---------- 4

Elementos importantes de la aplicación —---------- 4

Analizador Léxico —---------- 4

- Palabras Pertenecientes al Lenguaje —---------- 4
- Tokens —---------- 6
- Errores Léxicos —---------- 6

Controlador del Análisis —---------- 7

Gramática Libre de Contexto —---------- 7

Analizador Sintactico —---------- 8

- Funcionamiento —---------- 8
- Modo Pánico —---------- 9

Nodo Árbol —---------- 9

Árbol de Sintaxis Abstracta —---------- 9

Tabla de Símbolos —---------- 10

Analizador Semántico —---------- 10

- Evita variables de AGENTE duplicados —---------- 10
- Evita cualquier tipo de Variable duplicada —---------- 10
- Revisa EJECUCION de los AGENTE —---------- 10
- Solo le permite a EXPORTAR utilizar variables declaradas —---------- 11

Introducción

Objetivo

Otorgar al usuario el soporte y ayuda necesaria para que pueda hacer una correcta ejecución de la aplicación de java del estudiante Milton, que en esta ocasión es el archivo del proyecto 2 para el laboratorio de Lenguajes Formales y de Programación, en esta ocasión es un compilador para el lenguaje para generación de Promts “PromtTzal”.

Requerimientos

Tener una versión de java instalada
Tener disponible el ejecutable .jar proporcionado por el estudiante aunque el archivo también se encuentra disponible en el repositorio de GitHub https://github.com/miltonajxup/Lab_Lenguajes_SS2026.git

Ejecución:

Con el acceso al ejecutable (llamado: Proyecto2_Lenguajes_SS2026-1.0-SNAPSHOT-jar-with-dependencies.jar), se necesita una terminal abierta en la carpeta donde se encuentra el programa y al ingresar en la terminal lo siguiente:

```
java -jar Proyecto2_Lenguajes_SS2026-1.0-SNAPSHOT-jar-with-dependencies.jar
```

Podremos acceder a la aplicación y comenzar a utilizarla

Manual Técnico

Elementos importantes de la aplicación

Analizador Léxico	Hecho con el apoyo de JFlex, se definió los distintos tokens y caracteres que pertenecen al lenguaje
Analizador Sintáctico	Esta parte fué hecha a mano, funciona de tal forma que al encontrar un tipo de token específico, verificará si la secuencia de tokens que sigue es válida para el lenguaje
Árbol de Sintaxis Abstracta	Consiste en un objeto que almacena información como el valor además puede tener objetos del mismo tipo (hijos), que pueden almacenar la misma información
Analizador Sintáctico	Crea la tabla de símbolos, a su vez evita variables duplicadas y es quien revisa al momento de EJECUTAR, si lo que se quiere ejecutar existe
IDEPromt	Es la parte visual para el usuario y da acceso a la edición, creación y compilación de archivos

1. Analizador Léxico

El analizador léxico que se implementó está hecho con el apoyo del generador de analizadores léxicos, jflex, dicho analizador fue diseñado con un archivo jflex en el cual se define las expresiones regulares y los patrones que se reconocen el lenguaje y tienen un significado para el mismo.

1.1. Palabras Pertenecientes al Lenguaje

Lexemas reconocibles dentro de un archivo .pz

Terminal (Tipo de Token)	Lexemas que producen
MODELO	@modelo
ROL	@rol
FORMATO	@formato
AGENTE	AGENTE
CONTEXTO	contexto
VARIABLE	variable
EJECUTAR	EJECUTAR
EXPORTAR	EXPORTAR
PREGUNTAR	PREGUNTAR
GENERA	GENERA
RESUMIR	RESUMIR
ANALIZAR	ANALIZAR
TRADUCIR	TRADUCIR
CLASIFICAR	CLASIFICAR
EXTRAER	EXTRAER
CARGAR	CARGAR
SOBRE	SOBRE
DESDE	DESDE
EN	EN
COMO	COMO
ASIGNAR	->
ASIGNACION	=
CONCATENACION	+
CADENA	conjunto de caracteres delimitados por comillas dobles

ENTERO	números enteros positivos
IDENTIFICADOR	conjunto de caracteres que no son los lexemas antes definidos
PARENTESIS_IZQ	(
PARENTESIS_DER	)
LLAVE_IZQ	{
LLAVE_DER	}
COMA	,

// (Comentario de Línea: Ignora el texto que encuentre hasta encontrar un salto de línea)

/**/ (Comentario de Bloque: Al escribir /* se indica que si ignorará todo hasta encontrar el cierre con */)

Ignora los espacios en blanco, saltos de línea y tabulaciones

1.2. Tokens

La forma en que el analizador guarda la información de los elementos que coincidan con el lenguaje es de la forma:

“@modelo” { return new Token(tipo, lexema, fila, columna); }

En código queda:
"@modelo" { return new Token(TipoToken.MODELO, yytext(), yyline + 1, yycolumn + 1); }

1.3. Errores Léxicos

Los caracteres que no se reconocen se atrapan en forma de un error junto con su información

(Esquema ilustrativo, no es código)
. { return new Token(tipo, lexema fila, columna, descripción); }

2. Controlador del Análisis

El programa tiene una clase que controla los resultados de los analizadores, una vez pasado el archivo de código fuente, el analizador léxico regresa en forma de token los elementos que coinciden en forma de token, los guarda en listas y al final pasa la lista de tokens al analizador sintáctico

3. Gramática Libre de Contexto

A modo de simplificación en la tabla de secuencias, cuando en la tabla aparece directivas, comandos_ia o conector, hace referencia a uno de los tokens siguientes:

directivas: @modelo, @rol, @formato
comandos_ia: EJECUTAR, EXPORTAR, PREGUNTAR,GENERA, RESUMIR, ANALIZAR, TRADUCIR, CLASIFICAR, EXTRAER
conector: SOBRE DESDE EN COMO

Nota: En algunas secuencias se permite ε porque el compilador no tiene una regla semántica que restrinja que al menos tiene que existir 1

programa	->	directivas agentes ejecuciones exportar
directivas	->	DIRECTIVA CADENA directivas | ε
agentes	->	agente agentes | ε
agente	->	AGENTE IDENTIFICADOR LLAVE_IZQ contexto variables comandos_ia LLAVE_DER
contexto	->	CONTEXTO ASIGNACION CADENA contextos | ε
variables	->	VARIABLE IDENTIFICADOR ASIGNACION expresion variables | ε
comandos_ia	->	comando_ia comandos_ia | ε
comando_ia	->	COMANDO_IA expresion conectores ASIGNAR IDENTIFICADOR comando_ia | ε
conectores	->	CONECTOR expresion conectores | ε

expresion	->	termino mas_terminos
mas_terminos	->	CONCATENACION termino mas_terminos | ε
termino	->	CADENA | ID | ENTERO | unidad | cargar
unidad	->	IDENTIFICADOR | ε
cargar	->	CARGAR PARENTESIS_IZQ CADENA PARENTESIS_IZQ
ejecuciones	->	EJECUTAR IDENTIFICADOR mas_ejecuciones | ε
mas_ejecuciones	->	COMA IDENTIFICADOR mas_ejecuciones | ε
exportar	->	EXPORTAR IDENTIFICADOR mas_ids
mas_ids	->	COMA IDENTIFICADOR mas_ids | ε

4. Analizador Sintáctico

El analizador sintáctico es el encargado de revisar que los tokens que se reconocieron y que fueron guardados en el orden que fueron apareciendo, cumplan con las reglas que se establecen en la gramática libre de contexto.

4.1. Funcionamiento

El analizador tiene tokens predeterminados que se espera que inicien las secuencias, como lo es AGENTE, cuando un token coincide con alguno de estos tokens predeterminadas, se inicia el reconocimiento de la secuencia

- Se avanza
- Se revisa si coincide con el token que se espera
- Continúa hasta terminar con la secuencia y se reinicia el proceso

Aunque existen tokens como AGENTE, que se espera que almacene secuencias internas con su propio comportamiento, entonces al entrar a un AGENTE, se inicia un proceso igual al anterior en donde se esperan tokens predeterminados que inician secuencias y al encontrar cada secuencia tiene un comportamiento específico para ese tipo de token.

El proceso continúa hasta terminar de revisar los tokens encontrados por el analizador léxico.

4.2. Modo Pánico

El modo pánico implementado se lleva a cabo al momento de verificar el token esperado, si el token esperado no coincide con token que se está leyendo (token actual) entonces es un error y se consumen tokens hasta encontrar uno de sincronización, estos pueden ser AGENTE, uno de los COMANDO_IA, LLAVE_DER, EJECUTAR o EXPORTAR o cuando se terminan los tokens reconocidos, entonces ser reporta el error y se reinicia el proceso, ya sea dentro de un AGENTE o en el espacio GLOBAL.

El modo pánico se detiene hasta encontrar un token de sincronización, pero no consume ese token, más bien lo deja en espera para ser reconocido y revisar si genera una secuencia válida.

El criterio al momento de definir qué se considera un error, es que en el espacio GLOBAL, solo acepta tokens de tipo directivas, agentes, ejecuciones y exportaciones como secuencias válidas de inicio y dentro de un agente, los tokens que inician una secuencia válida son variables, el contexto o comandos IA.

Existe una variante del modo pánico, al iniciar una secuencia, se cuenta con un número específico de tokens esperados, si no coincide se activa el modo pánico y de igual forma consume tokens hasta encontrar un token de sincronización.

5. NodoArbol

Esta clase es una clase que almacena información como: un tipo de token, un valor y puede o no tener una lista objetos esta misma clase, estos objetos de sí mismo son llamados hijos.

La idea es centralizar un punto de partida y que desde ese punto se comience a leer las instrucciones del programa.

6. Árbol de Sintaxis Abstracta

Al momento que se está analizando que la secuencia sea correcta, de ser correcta, se van creando objetos de nodoArbol que almacenan la información de las secuencia de tokens correctos, haciendo que un nodo importante guarde

información de la secuencia que le sigue a modo que sean sus nodos hijos, si estos hijos resulta ser importante porque puede almacenar información de una sub secuencia, también almacenará información de los hijos y así sucesivamente.

7. Tabla de Símbolos

La tabla de símbolos se comienza a construir junto con el reconocimiento de la secuencia de tokens (Gramática libre de contexto), de modo que cuando se encuentra un token VARIABLE o una variable que se crea a partir del token ASIGNAR(->), se agrega a una lista donde se almacenan estos símbolos.

8. Analizador Semántico

El analizador semántico consiste en la revisión de que los símbolos y el árbol tenga sentido aunque el analizador semántico implementado es básico cumple con funciones de este tipo.

8.1. Evita variables de AGENTE duplicados

Al momento que se declara un agente, se guarda el nombre nombre utilizado para su identificador, si posteriormente se declara una variable con el mismo nombre, el símbolo no se agrega y reporta el error.

8.2. Evita cualquier tipo de Variable duplicada

Principalmente se evita porque al momento de de instrucción EXPORTAR, puede quedar ambigüedad de cuál variable es a la que se refiere, porque no hay una forma de especificar a cuál variable se está refiriendo como:
agente1.variable1 y agente2.variable1

8.3. Revisa EJECUCION de los AGENTE

Los agentes declarados se guardan y al momento de la instrucción EJECUCION, busca los AGENTE que se declararon con anterioridad, si no encuentra el agente que se intenta ejecutar, entonces se reporta el error.

8.4. Solo le permite a EXPORTAR utilizar variables declaradas

Al momento de detectar una variable, se guarda esa información y al momento de la instrucción de exportar se revisa que tipo de variable se quiere exportar, si no coincide con una de las variables guardadas, entonces se reporta el error.